

JWT Authentication API

Spring Boot • Spring Security • JWT • PostgreSQL

Backend Authentication System built with Spring Boot

Technologies

- Java 21
- Spring Boot
- Spring Security
- JWT Authentication
- Spring Data JPA
- Hibernate
- PostgreSQL
- Docker
- Maven
- RESTful APIs
- Postman
- Swagger UI

auth-controller

POST /api/auth/register

GET /api/auth/me

GET /api/auth/login

user-controller

GET /users/admin

POST /users/admin

GET /api/public/users

DELETE /users/admin/{id}

Project Description

This project demonstrates a complete authentication system for REST APIs using JSON Web Tokens (JWT).

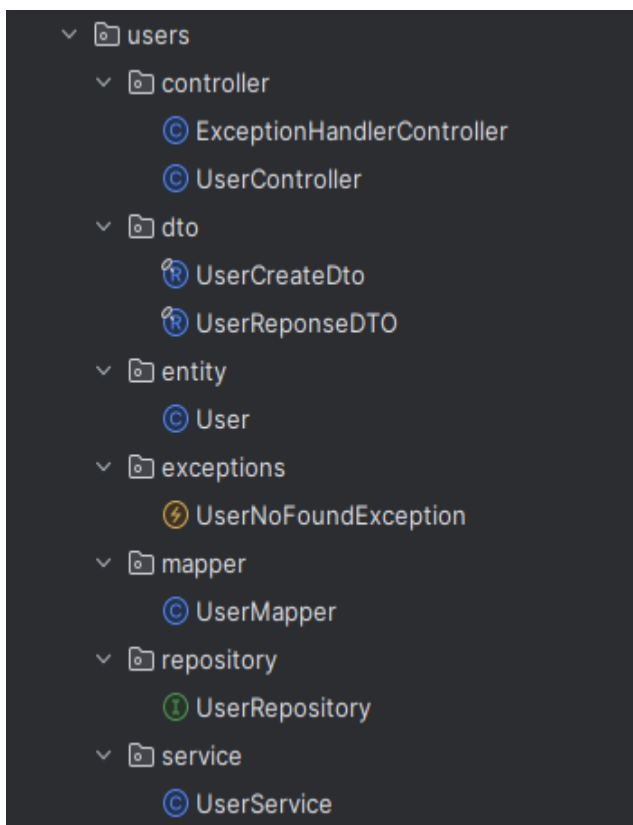
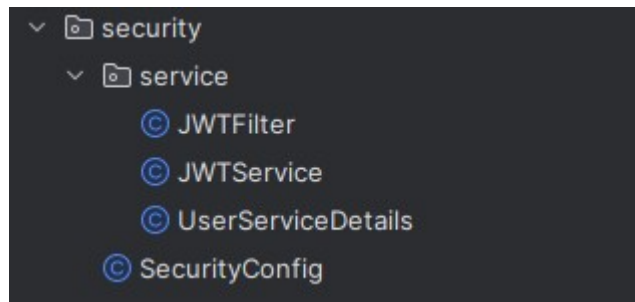
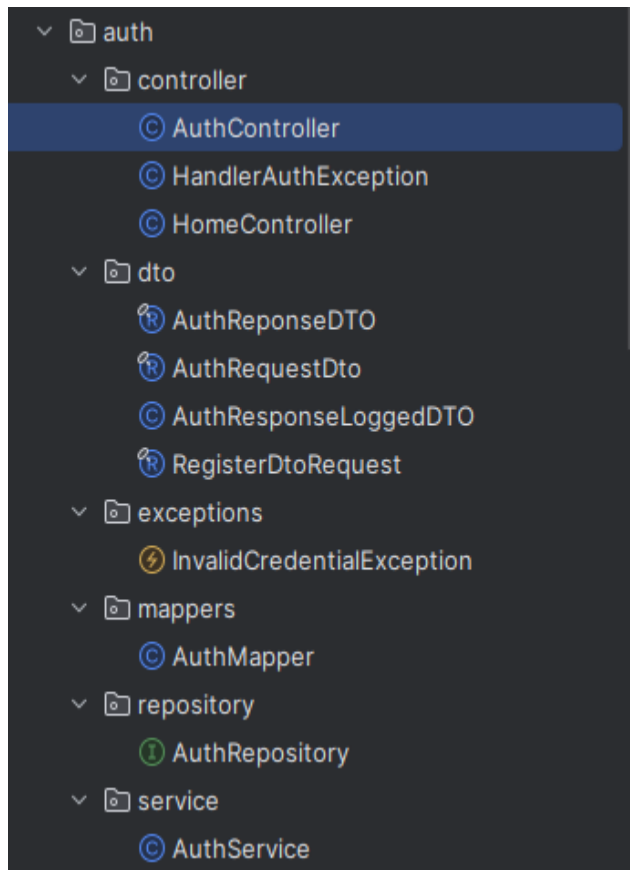
Implemented features include:

- User Registration
- User Login
- Password Encryption (BCrypt)
- JWT Generation
- JWT Validation
- Protected REST Endpoints
- Stateless Authentication
- Global Exception Handling
- DTO Pattern
- Layered Architecture

Project Goals

- Secure REST APIs
- Demonstrate Spring Security integration
- Follow clean architecture principles
- Separate business logic from authentication logic

Project Structure



Database

PostgreSQL running inside Docker

Tecnologías:

- PostgreSQL
- Docker Container
- Spring Data JPA
- Hibernate ORM

●	test_pgadmin	87b0115a2c7e	dpage/pgadmin4	8081:80 ↗
●	test_postgres_ddb1	961c662ec98b	postgres:15	5433:5432 ↗

base de datos en postgresql

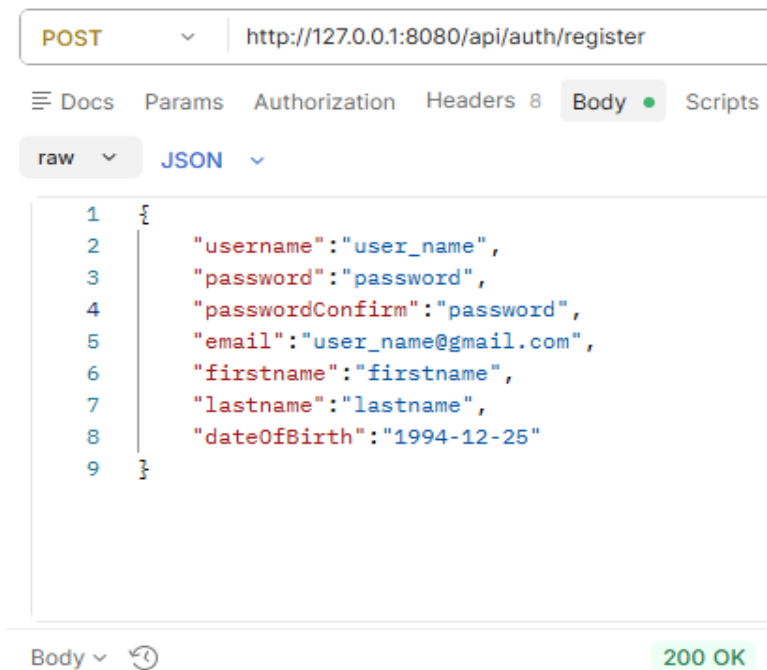
```
local_test=# select * from users;
```

user_id	date_of_birth	email
44816058-a2c4-4679-9277-c96dfb9283ee	1994-12-24 18:00:00	first@gmail.com
ca02c788-8e96-46a7-90fb-8ac19223fd27	1994-12-24 18:00:00	second@gmail.com
bcd8a60d-4e3e-456e-bad1-ad6f5f697ed4	2026-06-25 15:26:26.727	user1@gmail.com
7dd053a8-7960-434a-b92f-a42f03cde99d	1994-12-24 18:00:00	user_name@gmail.com

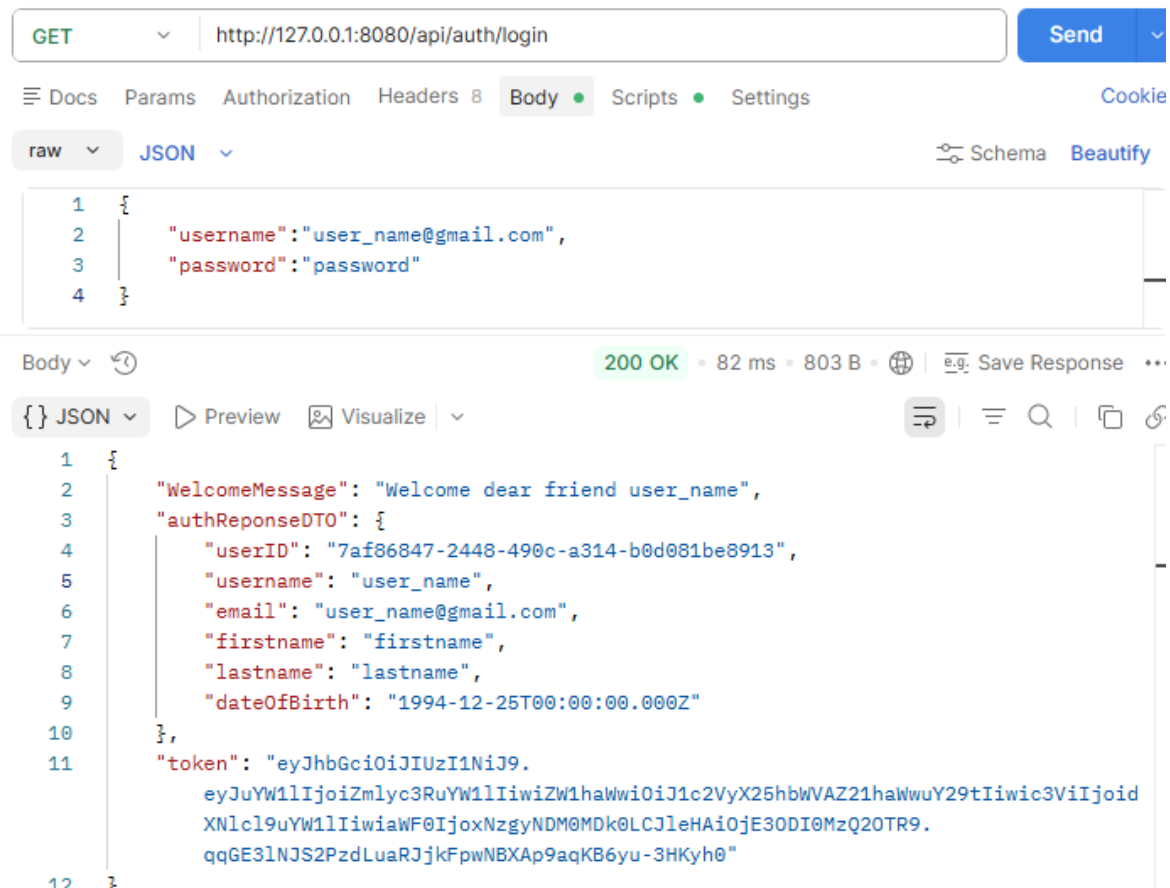
```
(4 rows)
```

API Demonstration

Register User: POST:- /api/auth/register



Login POST:- /api/auth/login



Protected Endpoints

login user: GET:- /api/auth/me bearer token

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8080/api/auth/me
- Auth Type:** Bearer ...
- Token:** {{token1}}
- Response:** 200 OK, 9 ms, 342 B. The response body is raw JSON:

```
1 user_name
```

login user: GET:- /api/auth/me no auth

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8080/api/auth/me
- Auth Type:** No Auth
- Response:** 403 Forbidden, 17 ms, 300 B. The response body is raw JSON:

```
1
```

Security Features

Implemented Security

- ✓ Password Encryption using BCrypt
- ✓ JWT Authentication
- ✓ Stateless Sessions
- ✓ Authentication Filter
- ✓ Security Context
- ✓ Spring Security Integration
- ✓ Protected REST Endpoints
- ✓ Exception Handling
- ✓ Token Validation